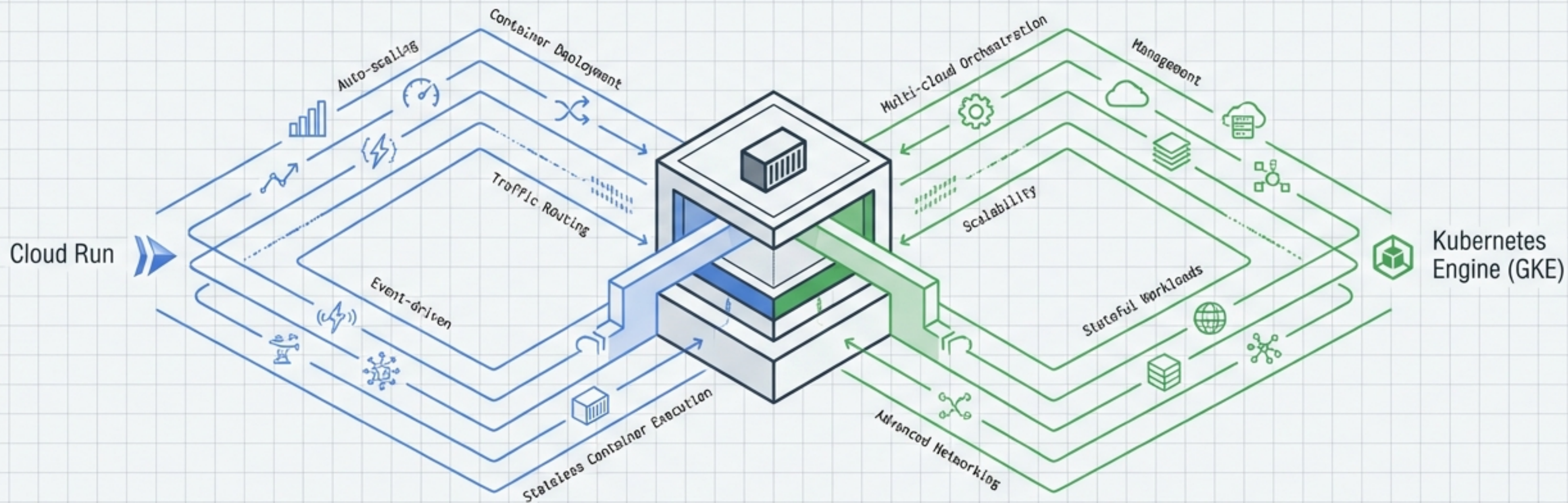


Modern Container Architecture on Google Cloud

A Strategic Engineering Guide to Cloud Run and Kubernetes Engine (GKE)



The Goal:

Run packaged, consistent,
containerized workloads in the cloud.

The Serverless Path

Technology:

Cloud Run

Philosophy:

Focus purely on code. Zero
infrastructure management.

Mechanism:

Stateless containers executed via web
requests or pub/sub events.

The Orchestration Path

Technology:

Google Kubernetes Engine (GKE)

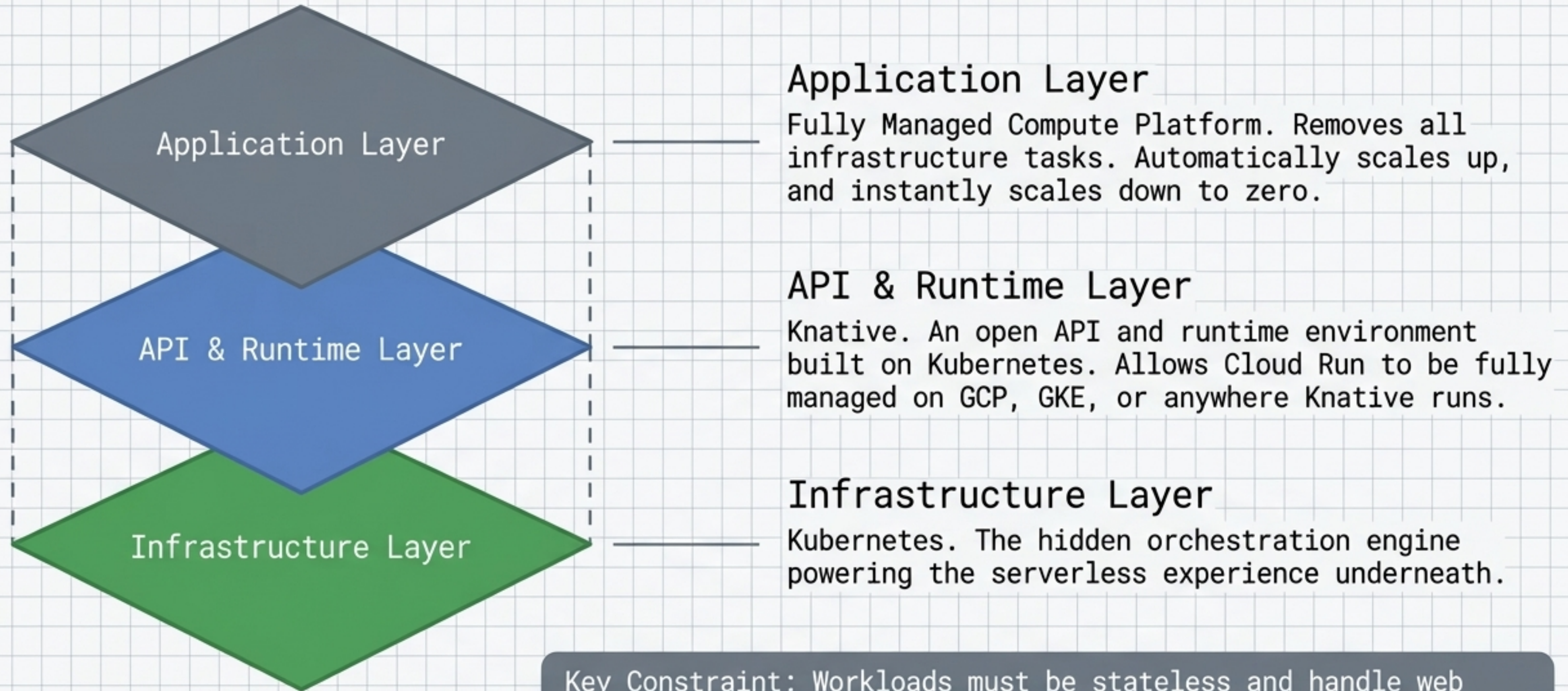
Philosophy:

Deep architectural control and
microservice coordination.

Mechanism:

Managed clusters handling complex
deployment rollouts and multi-container
pods.

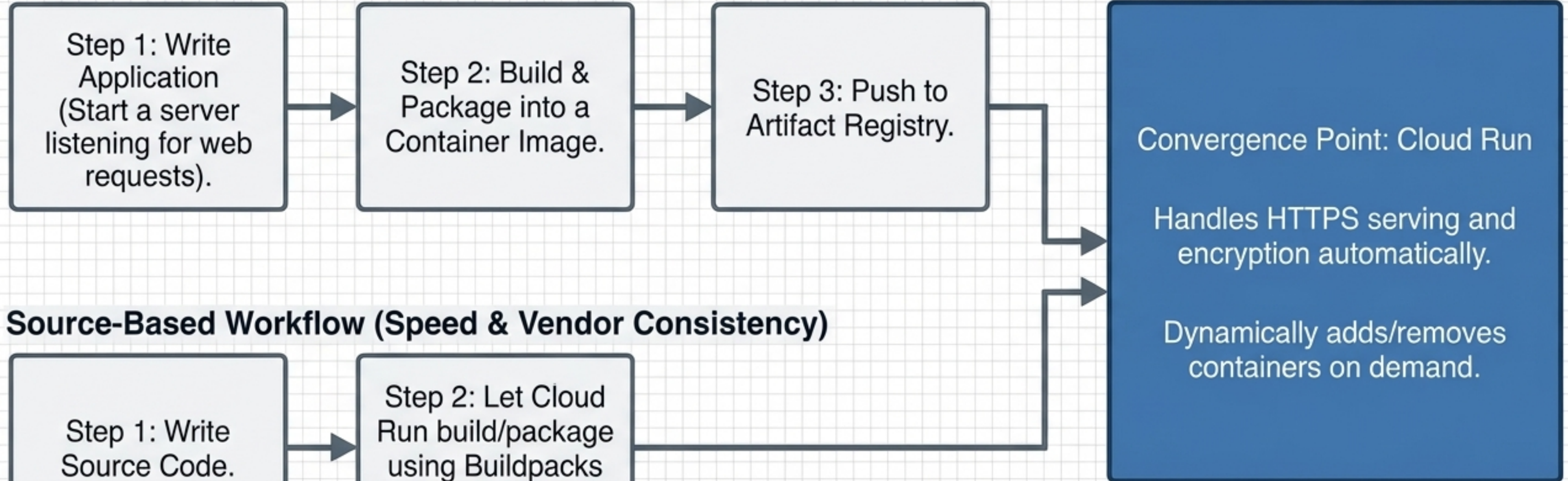
Cloud Run: The Serverless Blueprint



Key Constraint: Workloads must be stateless and handle web requests. Can run any binary compiled for Linux 64-bit (Java, Python, Node.js, PHP, Go, C++, Cobol, Haskell, Pearl).

The Deployment Pipeline

Container-Based Workflow (Flexibility & Transparency)

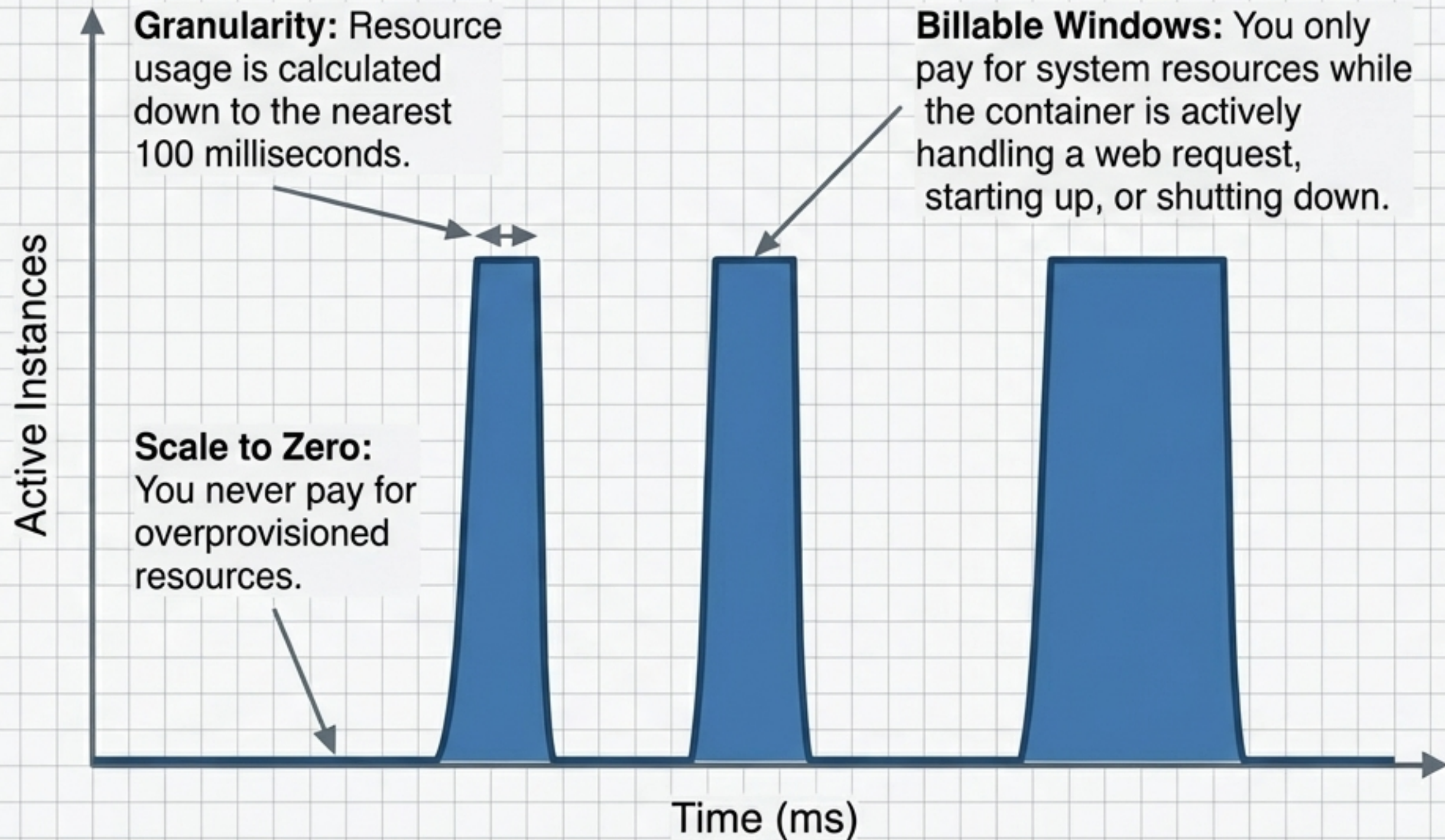


Source-Based Workflow (Speed & Vendor Consistency)



Output: Returns a unique HTTPS URL endpoint.

The Economics of Milliseconds

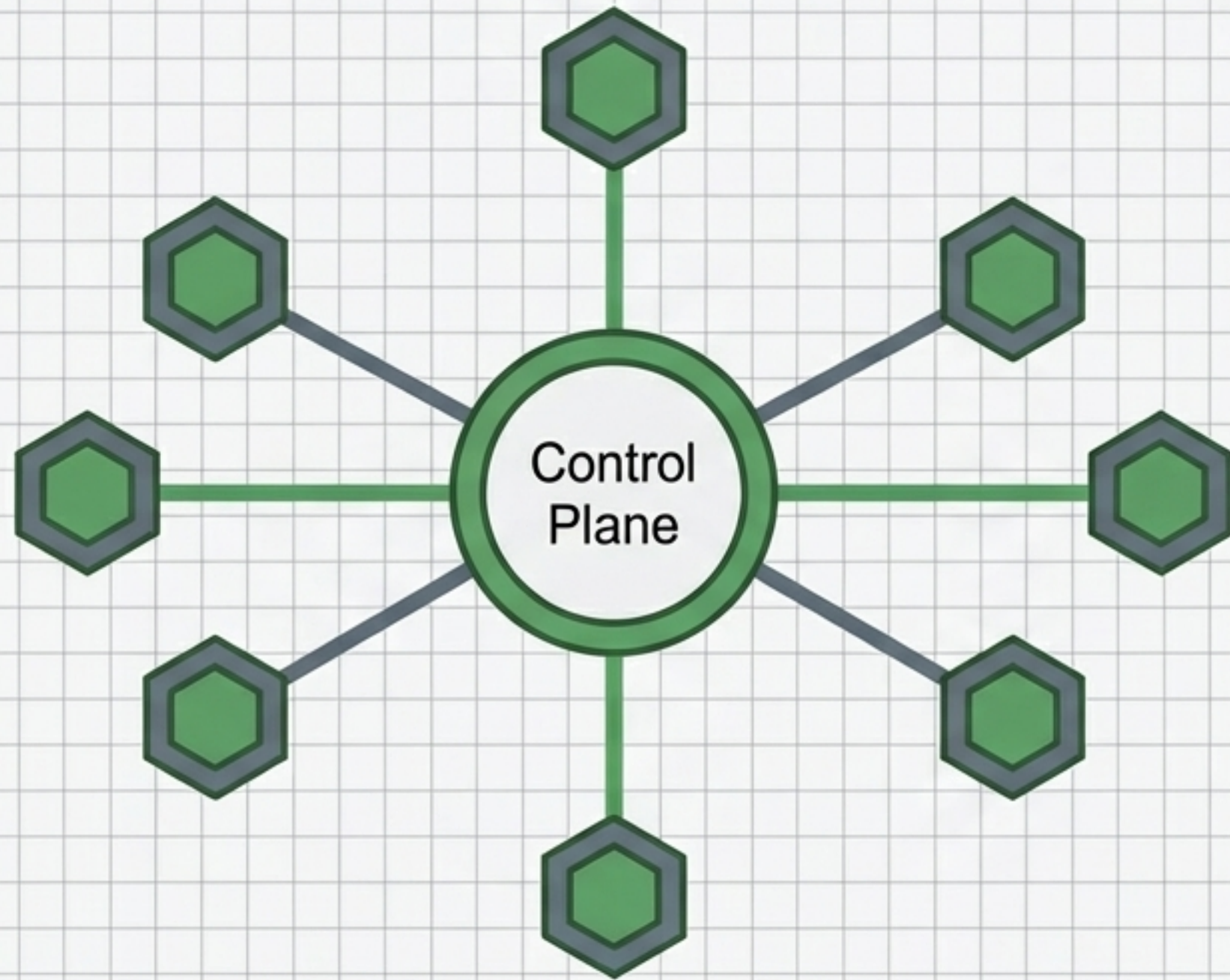


Cost Vectors:

- **Base fee:** Small fee per 1 million requests served.
- **Hardware scaling:** Price of container time increases with allocated vCPU and memory.

Kubernetes: The Orchestration Engine

Definition: An open-source platform for managing containerized workloads and services.



Orchestration

Coordinate many containers across many distinct hosts.

Abstraction

A set of APIs used to deploy containers on a cluster of nodes.

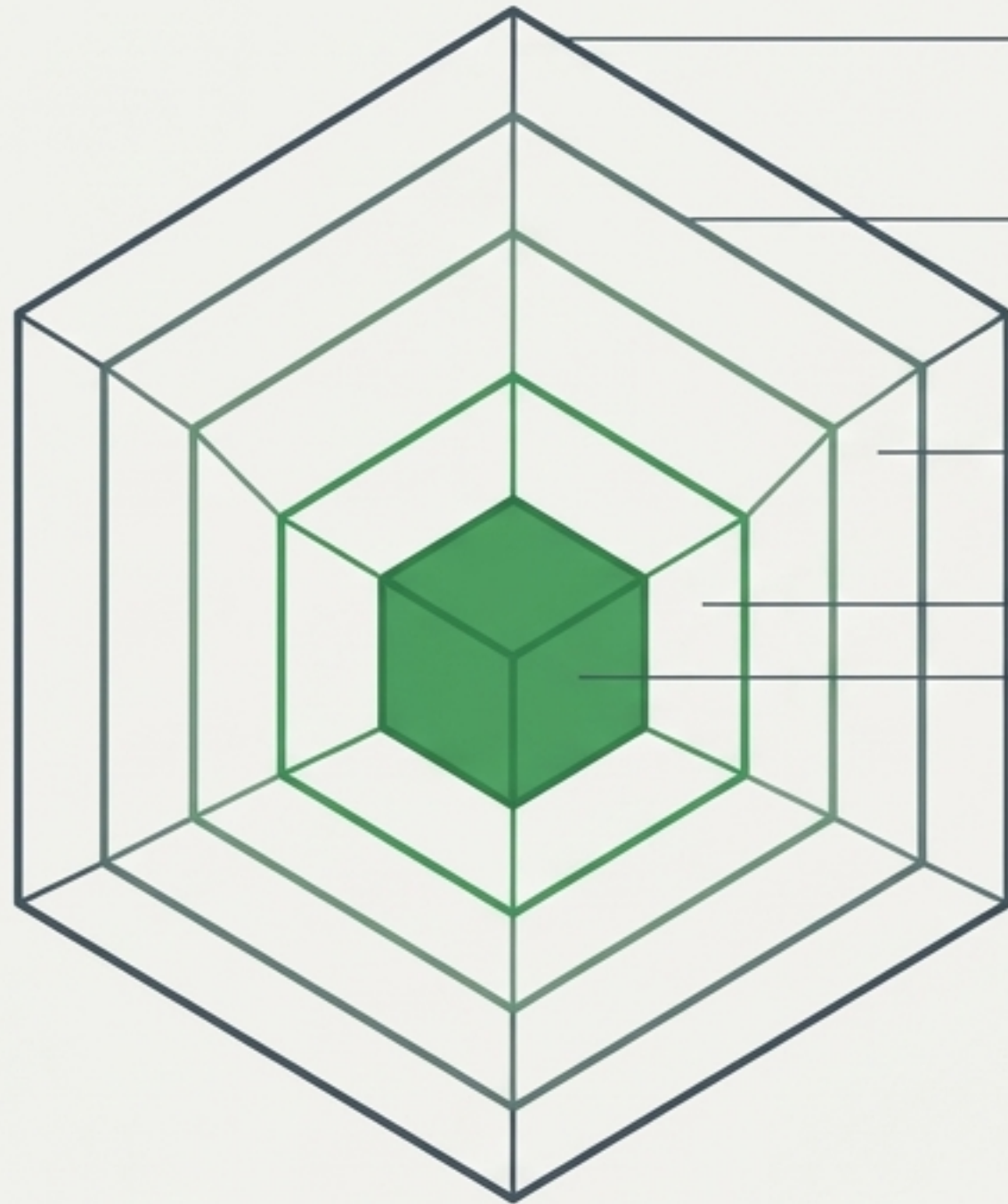
Scaling

Scale distinct workload components as independent microservices.

Lifecycle Management

Easily deploy sophisticated rollouts and rollbacks.

Anatomy of a Cluster



Level 1: The Cluster

The total system environment (Control plane + Worker Nodes).

Level 2: The Node

A computing instance (e.g., a virtual machine running in Compute Engine).

Level 3: The Deployment

Represents a group of replicas of the same pod. Keeps pods running even if underlying nodes fail.

`kubectl get deployments`

Level 4: The Pod

The smallest deployable unit in K8s. A wrapper around one or more containers. Provides a unique network IP.

`kubectl get pods`

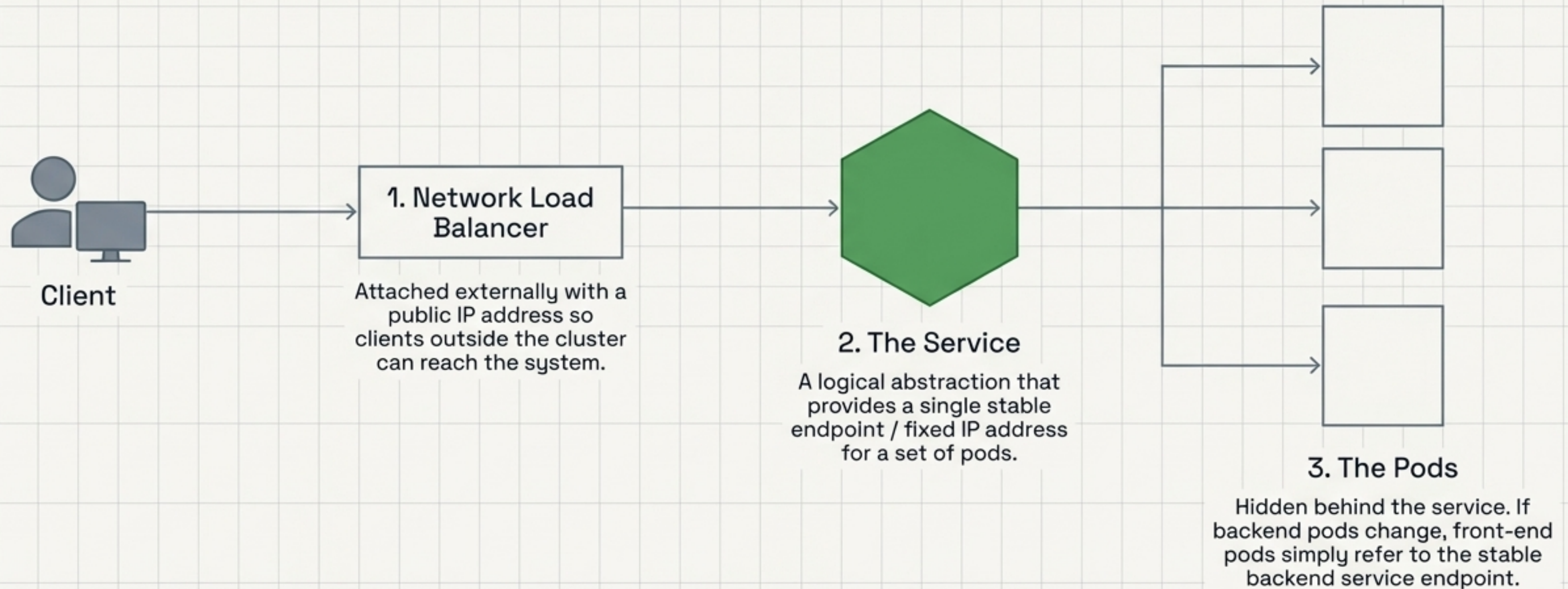
Level 5: The Container

The actual running application process. Multiple containers with hard dependencies can share a pod's network and storage.

Directing Traffic: Services and Load Balancers

The Problem:

Deployments create and destroy pods constantly. Pod IP addresses are ephemeral and don't remain stable over time.



Managing State: Imperative vs. Declarative Control

Imperative (Manual Steps)

Concept: Issuing direct, step-by-step commands. Great for learning and testing.

Examples: `kubectl run`, `kubectl scale`, `kubectl expose`.

Autoscaling: Can be configured to increase pod count based on parameters like CPU utilization.

Declarative (Desired State)

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: my-deployment
spec:
  replicas: 5
  selector:
    matchLabels:
      app: my-app
  template:
    metadata:
      labels:
        app: my-app
    spec:
      containers:
        - name: my-container
          image: nginx:latest
```

Concept: The true strength of K8s. You provide a configuration file outlining the desired final state; K8s figures out how to make it happen.

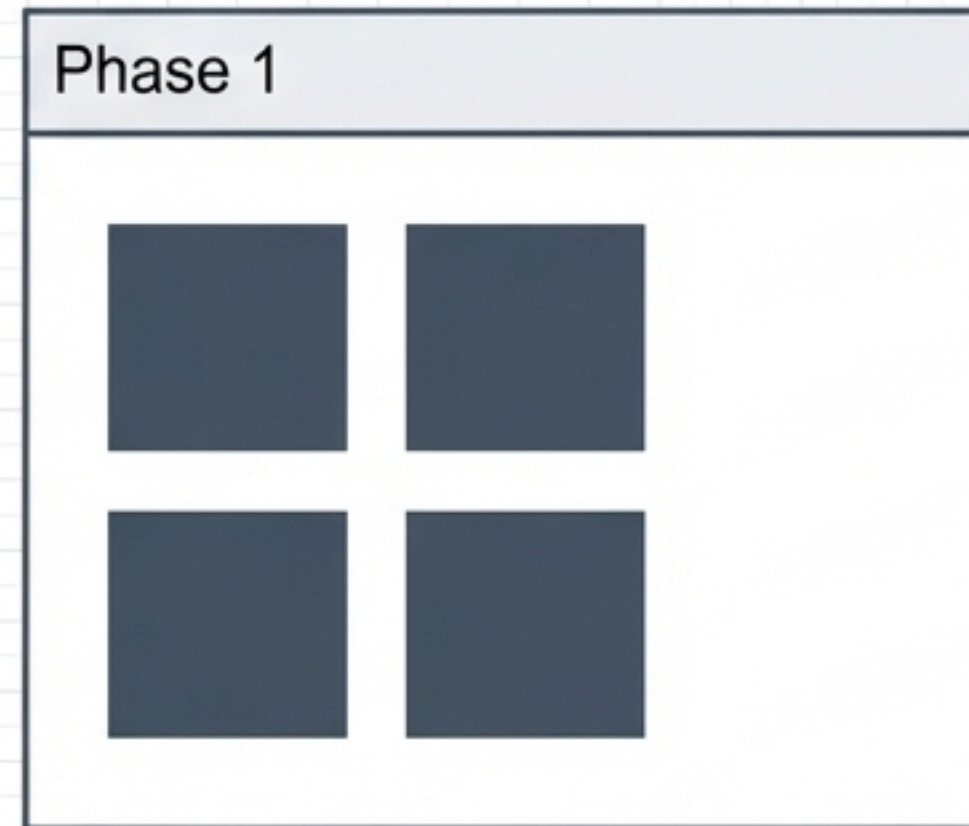
Workflow: Update the deployment config file (e.g., change replicas from 3 to 5), then execute:

```
kubectl apply
```

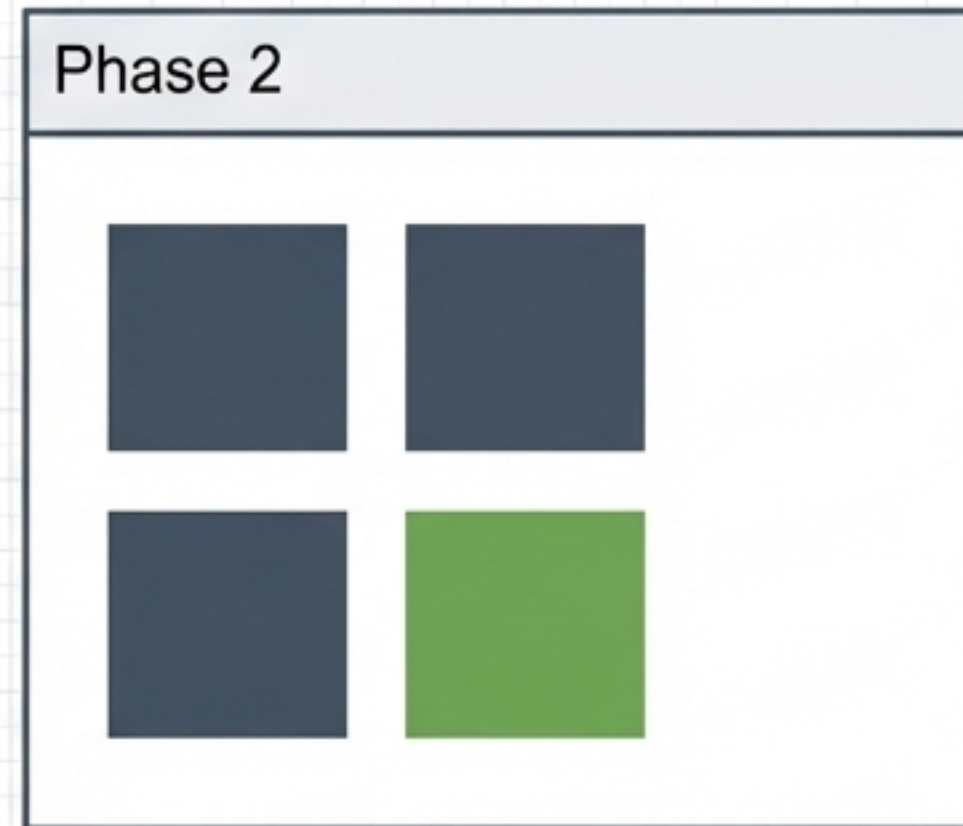

Zero-Downtime Rolling Updates

The Challenge: Updating a container to get new code in front of users simultaneously is risky.

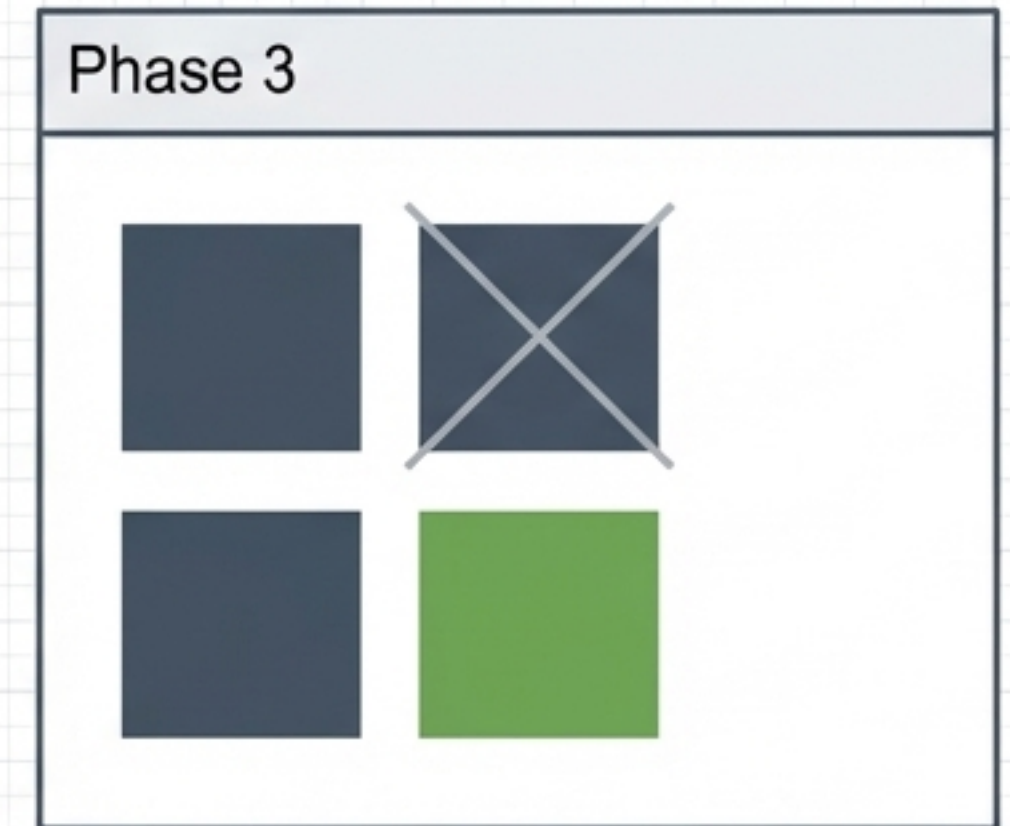
The Mechanism: Use `kubectl rollout` or update the config and `kubectl apply`.



Cluster runs pods on V1.



K8s creates a new individual pod on V2.

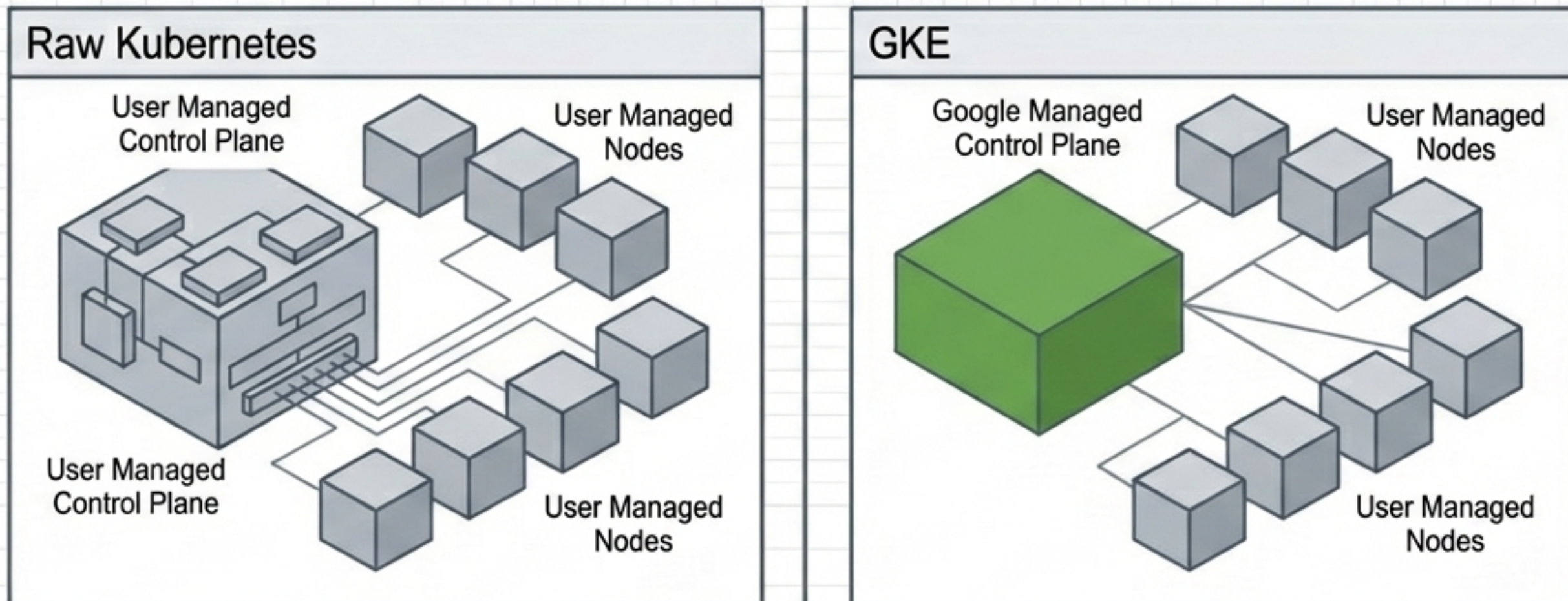


K8s waits for the new V2 pod to be available and healthy before destroying one of the old V1 pods. Process repeats until fully updated.

Google Kubernetes Engine (GKE)

The Managed Advantage: GKE eliminates the need for a separate control plane. Google manages all control plane components, exposing a single IP address for Kubernetes API requests.

Before and After



`gcloud container clusters create k1`

Advanced Cluster Management Features:

- Native GCP Load Balancing for Compute Engine
- Node Pools (subsets of nodes for flexibility)
- Automatic Scaling (cluster node instance count)
- Node Auto-repair & Auto-upgrades
- Observability logging and monitoring

GKE Operating Modes: Autopilot vs. Standard

Autopilot (Recommended)	Standard (Customization)
Infrastructure: GKE manages underlying infrastructure completely.	Infrastructure: User manages the underlying infrastructure.
Automated Features: Handles node configuration, autoscaling, auto-upgrades, baseline security, and networking configuration.	Automated Features: User is responsible for configuring individual nodes, managing, and optimizing the cluster.
Value: Optimized for production, promotes a strong security posture, and maximizes operational efficiency.	Value: Required only when highly specific levels of granular configuration control are needed.

The Ultimate Architectural Decision Framework

Cloud Run (The Serverless App Focus)

Best For:

Stateless applications, event-driven web requests.

Overhead:

Zero infrastructure management. Focus entirely on code.

Scaling & Cost:

Instant scale to zero. Billed down to 100ms granularity only during active request handling.

GKE (The Orchestration Focus)

Best For:

Complex microservices, multi-container pods, workloads requiring granular network/state control.

Overhead:

Managed control plane, but requires cluster design (Standard) or workload configuration (Autopilot).

Scaling & Cost:

Cluster-level autoscaling. Resource allocation based on node pools.

Build For The Workload, Not The Infrastructure



Google Cloud's container ecosystem offers a seamless spectrum of control.

Whether shifting operational burden entirely to the platform with **Cloud Run** or designing complex, self-healing microservice ecosystems with **GKE**, the foundation is identical: open-source, flexible, and strictly containerized.

Architectural Mandate: Start with the workload constraints, map the necessary level of control, and deploy.